CSD-CoKV: Host-CSD Collaborative Offloading for High-Performance LSM-tree based KV Stores

Zhining Cao†, Kai Zhang‡, Jinrun Yang†, Hui Li‡, Nan Su‡, Qian Wei†, Shikun Ma†, Zehao Chen*, Junbo Yin‡, Haijun Zhang‡, Zhaoyan Shen†

†School of Computer Science and Technology, Shandong University, Qingdao, China

*School of Cyber Science and Technology, Shandong University, Qingdao, China

‡Inspur (Jinan) Data Technology Co., Ltd, Jinan, Shandong, China

***Abstract*—LSM-tree-based key-value stores are widely used due to their write efficiency and scalability. However, their background compaction process requires substantial data movement between the host and storage devices, resulting in inefficient resource usage, frequent write stalls, and degraded overall throughput. Existing optimizations fail to address the performance bottlenecks caused by excessive system interconnect traffic. Although in-storage computing offers a promising way to mitigate data transfer overhead, prior works typically offload all compaction tasks blindly to computational storage devices. We have performed plenty of preliminary experiments and observed that such naive offloading leads to performance inefficiency due to CSD resource constraints.**

In this work, we propose CSD-CoKV, a novel framework that harmonizes CSD arrays with the host to accelerate LSM-tree compaction more intelligently. We propose a semantic-aware offloader to selectively pick appropriate compaction jobs to CSD arrays and deploys a lightweight high-speed compaction kernel inside each CSD. Additionally, we design a CSD-friendly data partitioning scheme that leverages both KV store characteristics and CSD storage locality to achieve balanced workload distribution and improved efficiency in CSD arrays. Evaluations show that, with the same number of compaction threads, CSD-CoKV improves write throughput by up to 2.39× and boosts compaction speed by up to 3.28×.

***Index Terms*—LSM-tree based KV Store, Computational Storage Device, Collaborative Offloading**

I. Introduction

Log-Structured Merge-tree (LSM-tree) [1] based key-value (KV) stores have become the backbone of modern data-intensive systems due to their efficient data management. By buffering writes in memory and periodically flushing them to disk in a sequential manner, LSM-trees significantly reduce random I/O and provide high performance for write-heavy workloads. As a result, they have been widely adopted in large-scale storage engines such as RocksDB [2], LevelDB [3], Apache Cassandra [4], and HBase [5], powering a range of real-world applications. However, despite these advantages, LSM-tree-based KV stores frequently suffer from write stalls, primarily caused by the background compaction operations. Compaction maintains data consistency and controls space amplification by merging existing SSTables (files used to store KV items) in different levels, but it incurs substantial data movement between disks and the host CPU. When compaction is backlogged, write operations are delayed or stalled, leading to significant degradation in overall throughput.

To address the challenges of write delay or stall caused by compaction operations in LSM-tree-based KV stores, two kinds of optimization techniques have been proposed. The first focuses on software-level tuning, which includes adjusting compaction strategies [6] or adopting KV separation [7] to improve compaction efficiency. These methods aim to accelerate compaction or reduce its computational overhead, thereby mitigating write stalls and improving overall system throughput. The second approach introduces hardware devices or accelerators to enhance performance. Some works [8]–[12] propose multi-tiered storage hierarchies based on non-volatile memory (NVM) and design NVM-specific optimizations. Others leverage FPGAs [13], [14], GPUs [15], [16] to offload compaction computations, thereby boosting performance. However, in both cases, the dominant performance bottleneck remains the large volume of data movement required by compaction, which is fundamentally constrained by the limited bandwidth of the PCIe interface. Thus, reducing the overhead and contention of data transfers between the host and storage remains a critical and unsolved problem.

With the emergence of computational storage devices (CSDs) [17]–[19] such as Samsung SmartSSD [20], which tightly integrates compute units within SSDs, there is an opportunity to offload parts of I/O-intensive workloads directly into CSDs. By establishing specific data paths between the SSD controller and the on-device compute logic, CSDs enables in storage computing for certain tasks and eliminate unnecessary data transfers between host memory and storage devices. Integrating CSDs into LSM-tree KV stores and offloading compaction workload into storage becomes a promising solution to mitigate system interconnect overhead and alleviate data transfer bottlenecks.

Although prior works [21]–[23] have proposed using CSDs to offload computations from KV Stores, these efforts have been implemented through emulation or have not been embedded into a real database. Moreover, their offloading strategies simply offloads all compaction operations to CSDs without considering the constraints in real devices. In practice, CSDs are resource-constrained embedded systems with lower computational power and memory capacity than general-purpose

Corresponding author: Zhaoyan Shen, School of Computer Science and Technology, Shandong University, Qingdao, China. E-mail: shenzhaoyan@sdu.edu.cn.

CPUs or dedicated accelerators. Blindly offloading compaction risks resource exhaustion (or resource contention) and performance degradation. Consequently, effectively integrating CSDs into LSM-tree-based KV stores presents significant challenges, raising three critical questions:

Q-1. How to fully exploit the computational resources within CSDs. Given the limited computation and memory resources of CSDs, designing an efficient kernel for executing compaction computations is of paramount importance under such resource constraints. A faster acceleration kernel will enable more opportunities for CSD to offload computations.

Q-2. How to pick suitable compaction jobs for offloading to CSD. Though with a highly optimized CSD kernel, not all compaction operations are suitable for in-storage execution. A lightweight, metadata-guided selection mechanism is needed to identify compaction jobs that are appropriate for offloading.

Q-3. How to effectively enhance performance scalability with CSD arrays. Although CSDs provide independent internal data paths for bandwidth scaling, compaction computations offloaded to CSDs may involve data that are not located within the same disk, preventing the effective utilization of the internal data paths for bandwidth expansion. This requires careful data placement and task balance strategies among CSD arrays to enable performance scalability.

To address these questions, we propose CSD-CoKV, a software/hardware co-design for accelerating LSM-based KV stores with CSD arrays. We first conduct a series of preliminary experiments with different hardware and software configurations to evaluate the performance variations of compactions and find two key observations. We find naively offloading all compaction jobs to CSDs can degrade overall system performance, and compaction tasks from lower levels or with low key-range overlap ratios exhibit poor performance when executed on the host. Based on the insights derived from these observations, we first design a semantic-aware offloader that achieves computation offloading and scheduling on CSDs by considering metadata information of compactions, such as their output levels and overlap ratio of the key range in the input files. Moreover, by analyzing the workflow of compaction, we identify data dependencies and computational characteristics within the process and design a high-speed FPGA-based acceleration kernel for compaction. Finally, by leveraging the inherent properties of LSM-tree-based KV stores and the isolated data paths in CSDs, we propose a CSD-aware data partitioning scheme that intelligently balances data distribution and computational workload across devices.

We deploy a full-featured prototype of CSD-CoKV on a real system by integrating the computation acceleration kernel into the Samsung's CSD product SmartSSD and embedding it into RocksDB. Extensive experiments with both micro- and macro-workloads demonstrate that, compared with the host-only design, the maximum improvements observed are up to 2.39 times in write throughput and 3.28 times in compaction read throughput. The prototype is publicly available at https://github.com/caozhining/CSD-RocksDB

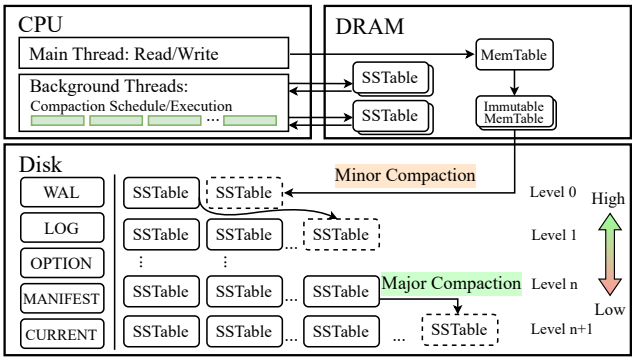


Fig. 1. The structure of LSM-tree based KV store.

II. Background

A. LSM-tree based KV Store

KV stores have been widely studied in both industrial and academic settings [24]–[27] due to their efficiency, scalability, and versatility across diverse workloads. Among them, LSM-tree based KV stores demonstrate superior performance in write-intensive scenarios and have been adopted by prominent systems such as LevelDB [3], RocksDB [2], and HBase [5].

As illustrated in Figure 1, the LSM-tree architecture optimizes write performance through a multi-level, write-optimized design. Incoming writes are first buffered in an in-memory structure called *memtable*, usually implemented as a skip list to preserve key ordering. Once a *memtable* reaches capacity, it is converted into an immutable *memtable* and asynchronously flushed to disk as an *SSTable* (Sorted String Table) with a process known as minor compaction. This append-only approach minimizes random I/O by leveraging sequential writes. However, the multi-level structure of LSM-trees can introduce read overhead. To mitigate this, modern KV stores employ auxiliary structures such as Bloom filters and index blocks, thereby maintaining read efficiency despite the write-optimized design.

While the write path of LSM-trees is optimized for high throughput, sustaining long-term efficiency requires compaction (also called major compaction), a critical background maintenance operation. Compaction merges *SSTables* across adjacent levels, ensuring global key ordering, reclaiming storage space by removing obsolete or deleted entries, and mitigating read amplification. A typical compaction job selects overlapping *SSTables* from two adjacent levels (e.g., Level 'n' and Level 'n+1'), reads them into memory, performs a multi-way merge sort, and writes the merged results back to disk at the next level. Compaction is essential for maintaining consistent query performance and storage efficiency over time. However, compaction introduces substantial computation and I/O overhead. The repeated merging and rewriting of data across levels leads to write amplification, where a single logical write may trigger multiple physical disk writes as data progresses to lower levels. Additionally, the large-scale data movement during compaction consumes significant I/O bandwidth, which interferess with foreground operations, resulting in unpredictable latency spikes and low performance.

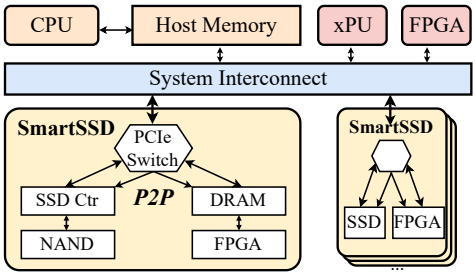


Fig. 2. Typical CSD architecture (e.g., Samsung SmartSSD)

To mitigate excessive compaction pressure, LSM-tree-based KV stores employ flow control mechanisms such as *write delay* and *write stall*. When pending compactions or the volume of data requiring compaction surpass predefined thresholds, typically derived from heuristics considering *memtable* states, Level 0 file counts, and overall compaction size, the system may delay incoming writes or temporarily halt them entirely. While these safeguards are critical for system stability, they introduce significant trade-offs: under sustained high-ingest workloads, they can severely degrade write latency and throughput. This limitation underscores the importance of developing more efficient compaction strategies. Consequently, optimizing compaction efficiency and overhead remains a fundamental challenge in LSM-tree-based KV store design.

B. Computational Storage Devices

In-storage computing has emerged as a promising approach to mitigate data movement bottlenecks in I/O-intensive systems [18], [19], [28]. Samsung's SmartSSD [20] represents one of the commercially available platforms that integrate programmable compute capabilities directly within storage hardware. As illustrated in Figure 2, the SmartSSD architecture couples an NVMe SSD with a lightweight FPGA accelerator through an internal PCIe switch. This design enables in-storage processing by allowing the FPGA to access SSD data directly, bypassing the host CPU and system interconnect. Critically, the FPGA communicates with the SSD via peer-to-peer (P2P) transfers, eliminating redundant host-side memory copies and reducing contention on external interconnects.

The SmartSSD architecture provides distinct PCIe address spaces for the SSD controller, FPGA logic, and FPGA-attached DRAM, enabling flexible programming models where the host can orchestrate computation, memory access, and storage I/O independently. Although individual SmartSSD bandwidth is constrained by its PCIe interface, deploying multiple devices in parallel yields linear scaling of both processing capacity and aggregate bandwidth. For I/O-intensive workloads, this architecture not only reduces host resource utilization and data movement overhead through computational offloading, but also enhances the effective bandwidth utilization of SSD arrays by leveraging near-data processing.

Typically, a SmartSSD integrates a lightweight FPGA (e.g., Xilinx Kintex™ UltraScale+ KU15P with 52,800 LUTs, 705,600 flip-flops, and 672 BRAM blocks, along with a 4 GB DDR4 DRAM) as its in-storage accelerator [29]. Compared to xPU-based accelerators or discrete FPGA cards, these resources are highly constrained. Consequently, designing efficient acceleration kernels is critical to fully exploit the limited capabilities of the CSD.

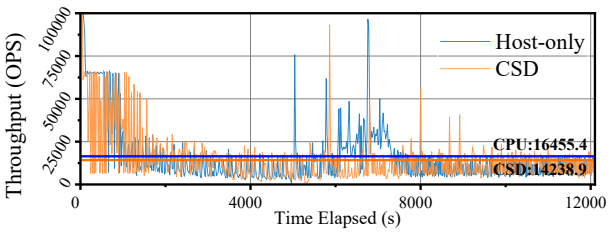


Fig. 3. Write throughput for writing 200 million key-value pairs.

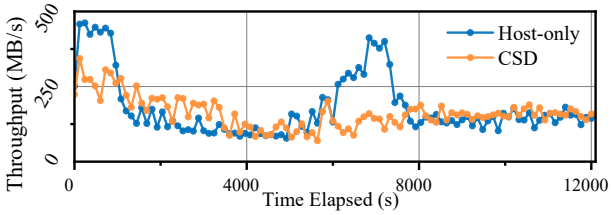


Fig. 4. Compaction throughput under CSD and host-only configurations.

III. Motivation

In this work, we aim to optimize LSM-tree performance by leveraging the computational and bandwidth advantages of CSDs to offload appropriate compaction operations. To better leverage the computational capabilities of CSD and identify suitable compaction jobs for offloading, we conduct a series of compaction-related preliminary experiments based on a typical LSM-tree based KV store, RocksDB. All experiments are performed using *db_bench* with the fillrandom workload to populate the database with uniformly distributed KV pairs. Our experimental configuration is the same as in Section V with only a single CSD. For the KV items, we adopt a representative fixed key size of 24 bytes and 1024-byte value size (referring to Facebook's social graph data [24]). Through these experiments and analysis, we derive two key observations.

Observation 1: Naively offloading all compaction jobs to CSD leads to performance degradation.

Prior works propose offload all compactions to hardware accelerators, such as in FPGA-based or xPU-based solutions, as these devices provide sufficient computational resources. In contrast, CSDs impose much tighter resource constraints, raising an open question as to whether this conventional strategy remains effective in such environments. To examine this, we implement a compaction acceleration kernel within the CSD (i.e. Samsung SmartSSD), adopting a full-offload configuration. For comparison, we configure the host with a single compaction thread on the same disk (with FPGA logic disabled) to ensure a fair baseline against the single compute engine within the CSD.

We populate the database with 200 million KV pairs. As shown in Figure 3, we present the write throughput along with the time elapsed for both the offloading-enabled (CSD) and the baseline (host-only). Although the offloading configuration exhibits higher throughput in certain time intervals, it results in an average throughput 13.5% lower than the baseline over the full execution period.

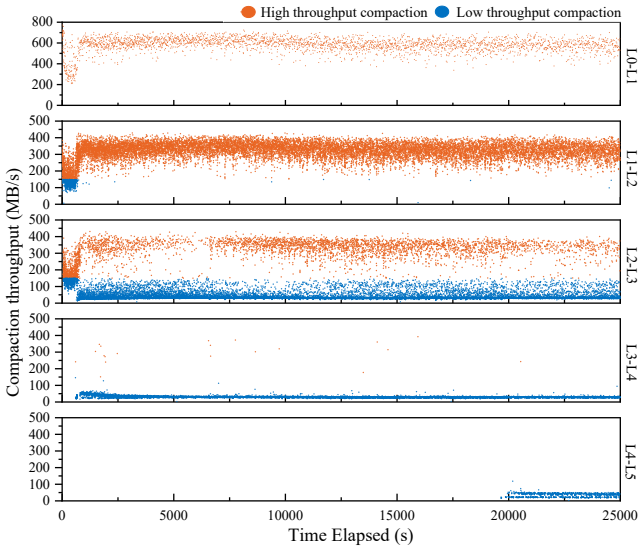


Fig. 5. Compaction throughput in different levels with 12 compaction threads in a host-only system (64GB DRAM). Compactions below the threshold of 150MB/s are marked in blue, while high-speed compactions above the threshold are marked in orange.

To further investigate the reason for this performance degradation, we analyze the compaction write throughput under both configurations. As shown in Figure 4, we find that the offloading strategy has a lower average compaction throughput (measured by dividing the total data compacted within each interval by the interval duration), although it achieves higher peaks during certain intervals. Comparing Figure 3 and Figure 4, we note a strong correlation between compaction throughput and overall write throughput, faster compaction effectively reduces write stall events, thereby improving ingestion performance. Therefore, due to its limited computational and memory resources, the CSD is unable to achieve efficient throughput when offloading all compactions, which in turn results in poor system throughput.

In summary, our results show that naively offloading all compaction jobs to CSDs leads to suboptimal compaction performance, which in turn causes reduced overall system throughput. However, the observation that certain time intervals exhibit improved performance suggests that some compaction jobs are better suited for offloading than others.

Observation 2: Compaction jobs targeting lower levels and involving low overlap key range ratios exhibit lower speeds on the host.

To investigate the reasons why integrating CSD leads to a degradation in overall performance and to identify compaction jobs that are more suitable for offloading to CSDs, we further conducted a series of comprehensive evaluations.

To emulate a realistic multi-thread CPU execution environment for KV stores, we enabled 12 background compaction threads. We begin by inserting one billion KV pairs of uniform size and then analyze the performance of all triggered compactions. The results are shown in Figure 5. Each point in Figure 5 represents a single compaction event, where the x-axis indicates its start time and the y-axis denotes the computed throughput. The evaluation results reveal a bimodal distribution in compaction throughput. To further analyze this pattern, we classify compactions into two categories: ”high-throughput” and ”low-throughput”, using a threshold of 150 MB/s. We set the threshold at 150 MB/s, which corresponds to the valley between the two modes in the observed bimodal distribution. Our observations indicate that compactions between L0-L1 and L1-L2 are predominantly fast, whereas L2-L3 compactions exhibit an alternating pattern of fast and slow compactions. In lower levels (L3-L4 and L4-L5), compactions are almost exclusively slow. Our analysis demonstrates a strong correlation between compaction speed and the target level in the LSM tree: compactions at lower levels are significantly more likely to be slower.

To investigate the root cause of this phenomenon, we analyze the compaction scheduling mechanism in KV stores. During compaction, a background thread sequentially traverses the LSM-tree levels from the highest to the second-lowest level. At each level, the thread evaluates compaction conditions to determine whether to initiate compaction. Once a level is selected and victim files are identified, the corresponding compaction job is constructed. Under write-intensive workloads, data in higher levels exhibits higher volatility and accumulates rapidly, increasing the likelihood of exceeding the compaction threshold and being selected. This design leads to higher-level compactions dominating the scheduling process, monopolizing system I/O and memory resources. Consequently, lower-level compactions, which inherently require more resources, are scheduled less frequently. When they do occur, they often experience resource starvation due to competing higher-level compactions, resulting in persistently low throughput for lower-level operations.

The evaluation results also indicate that input file characteristics substantially influence compaction performance. A compaction starts by selecting victim files (SSTables) from the start level (high level). Subsequently, files from the next level with overlapping key ranges are identified for inclusion. During this process, a score quantifying the key range overlap for each candidate file in the next level is calculated to determine selection priority: SSTables with lower overlap scores receive higher priority. The overlap is computed as:

$$\text{overlap score} = \frac{\text{overlapping_bytes} \times 1024}{\text{compensated_file_size} \times \text{ttl_boost_score}}$$

where *overlapping_bytes* represents the total size of overlapping regions, *compensated_file_size* accounts for the logical file size adjusted for compression, and *ttl_boost_score* incorporates time-to-live (TTL) effects that influence compaction priority. This selection mechanism inherently produces compactions with distinct input characteristics depending on the degree of key range overlap.

To evaluate the impact of file selection on system performance, we initialize the database with 400 million KV pairs and manually trigger compactions with varying overlap scores. As shown in Figure 6, we observe a strong correlation between overlap scores and compaction efficiency: higher

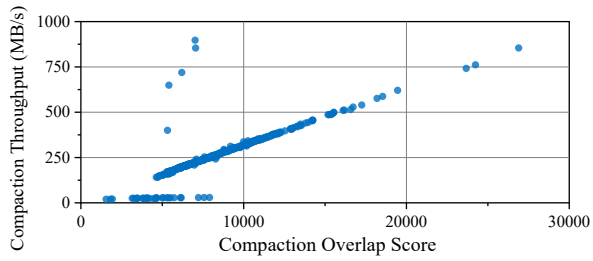


Fig. 6. Compaction throughput with different key range overlap ratios

scores correspond to better performance, while lower scores result in lower throughput. A higher overlap score indicates that the file overlaps with more SSTables in the next level, which allows large sequential runs to be processed together, reducing iterator switching overhead compared to fragmented overlaps and improving I/O efficiency. Conversely, a lower overlap score results in fewer overlapping SSTables, causing frequent context switches during merging and leading to inefficient compactions. Notably, when executing these same compactions on CSDs, performance remains consistently stable regardless of overlap score variations.

In summary, the above analysis suggests that compactions across different levels of the LSM-tree exhibit distinct characteristics. Higher-level compactions usually have higher overlap scores, better exploit available compute resources, and therefore achieve greater efficiency. By contrast, lower-level compactions are largely I/O-intensive, suffering from severe resource contention and frequent I/O stalls, which substantially degrades throughput. Despite the limited compute and memory capacity of CSDs, such inefficient compactions, either due to low overlap scores at higher levels or I/O-bound execution at lower levels, are particularly amenable to offloading. By leveraging their advantage in minimizing data transfer overhead, CSDs can effectively mitigate host-side bottlenecks and enhance the overall performance of KV stores.

IV. Design of CSD-CoKV

To harness computational storage devices for accelerating key-value stores, we present CSD-CoKV, a hardware-software co-designed system architecture. This section details our design in four parts: First, we provide an overview of the system architecture and describe its software stack implementation. Second, we introduce a semantic-aware offloader, whose design is guided by the key insights from Section III. Third, we present our FPGA-based accelerator for CSDs that achieves high-throughput compaction execution. Finally, we propose a CSD-friendly data partitioning scheme that ensures efficient performance scaling across CSD arrays.

A. System overview

Figure 7 illustrates the system architecture of CSD-CoKV. On the host side, we introduce a semantic-aware offloader to address the computational constraints of CSDs. When triggered, the background compaction thread dynamically schedules jobs for CSD offloading based on host compaction throughput and other system metrics.

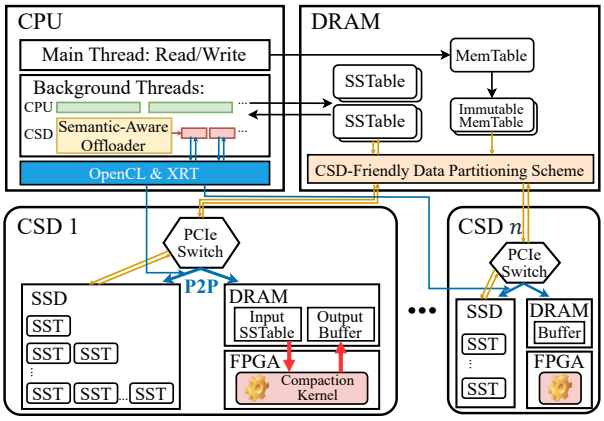


Fig. 7. CSD-CoKV architecture overview.

To optimize data management, we propose a CSD-friendly data partitioning scheme that acts as an intermediate layer between the KV store and disk storage. This scheme (1) determines file placement in the CSD array based on data distribution patterns during compaction and (2) records these mappings in file metadata, ensuring balanced data and workload distribution across CSDs. Section IV-D provides a detailed discussion of this partitioning strategy.

The CSD integrates computing capabilities alongside its SSD components, where we deploy a lightweight compaction acceleration kernel. The host system orchestrates data transfer between the CSD's computation accelerator and its SSD storage. During offloaded compaction execution, the host: (1) controls the transfer of input SSTable files to the CSD accelerator's memory via the host software stack, (2) collects and packages relevant SSTable metadata to guide the computation, and (3) initiates kernel execution on the CSD. Upon completion, the accelerator returns output SSTable metadata to the host, which subsequently updates its memory state and manages the writing of result files to persistent storage.

B. Semantic-Aware Offloader

Building on the compaction characteristics identified in Section III, we propose a Semantic-Aware Offloader to intelligently delegate compaction jobs to CSDs. The offloader comprises three key components: (1) a Profiler that identifies offloading levels and overlap score ranges, (2) a CSD thread allocator, and (3) a Compaction Picker that selects suitable jobs for CSD execution.

Our Profiler design is guided by Section III's findings that lower-level compactions and those with lower key-range overlap scores benefit most from CSD offloading. The Profiler maintains an in-memory buffer caching host-executed compaction metadata, including execution speeds, levels, and overlap scores. After each host compaction, the Profiler: records the execution metrics, computes moving averages (window size=10) for each level's compaction speed, derives level-specific overlap score thresholds.

A compaction is suitable for offloading if its level's average speed falls below the CSD execution speed threshold or its overlap score is below the level's overlap score threshold. This

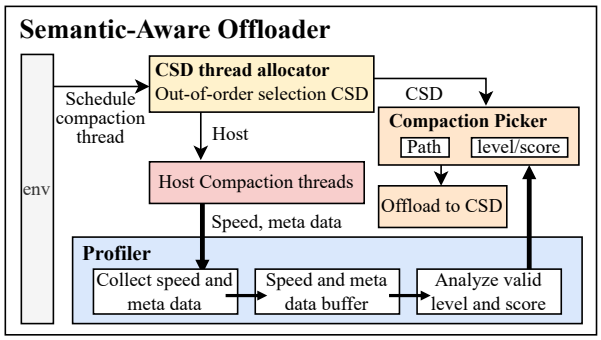


Fig. 8. Architecture of Semantic-Aware Offloader.

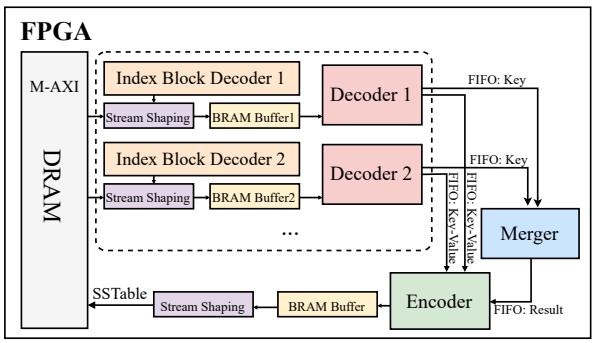


Fig. 9. Architecture of the Compaction Acceleration Kernel.

dual-threshold approach ensures efficient job selection while accounting for both computational and I/O characteristics.

To effectively distribute tasks across the CSD array, we introduce a CSD Thread Allocator that activates after compaction scheduling. The allocator employs an out-of-order selection policy to assign compaction threads to idle CSDs, ensuring workload balance. When no idle CSD is available, the system defaults to host execution.

Once a CSD is selected as the computing unit for the compaction thread, we use a Compaction Picker to select the files included in the compaction for CSD. The Profiler obtains the range of offloading levels and overlap scores boundary by collecting and calculating host compaction information. We select the files for the compaction based on these ranges and the path of the selected CSD. During the selection process, we traverse each level from higher to lower levels. If the corresponding level is not an offloading level, we select SSTables that meet the overlap score and are located on the corresponding CSD as the input files for this compaction; if the corresponding level is an offloading level, we select SSTables located on the corresponding CSD as the input files for this compaction. After the files are selected, we also check whether the selected compaction meets the resource requirements of the CSD. If it does, the compaction is offloaded to the CSD for computation; if not, it is returned to the host for computation.

In summary, our Compaction Scheduler employs intelligent job selection to optimize near-storage processing, strategically offloading compaction tasks that benefit most from CSD execution. This design achieves dual objectives: (1) maximizing computational storage resource utilization while (2) improving overall system throughput through balanced host-CSD workload distribution.

C. Compaction Acceleration Kernel in CSDs

While prior research has demonstrated FPGA-based compaction acceleration on discrete devices with abundant resources, Computational Storage Drives (CSDs) impose stringent computational and memory constraints that limit parallelism. To address this, we propose a lightweight acceleration kernel specifically optimized for CSD environments. Departing from the conventional strategy of scaling performance with multiple compute units, an approach infeasible due to the limited DRAM for staging SSTables on CSDs, our design maximizes the efficiency of a single unit through aggressive intra-kernel parallelism and resource-aware optimization. Specifically, we employ widened data paths, deep pipelining, and a balanced allocation of BRAM between buffering and computation. This "scale-up" philosophy, which results in over 80% BRAM utilization to fully exploit on-chip resources, distinguishes our work and makes compaction acceleration practical under CSD constraints.

The compaction process involves decoding KV pairs from input SSTables, performing sorting and merging operations to generate updated results, and finally re-encoding the data back into the SSTable format before writing it to storage. To support this workflow, our design incorporates three primary modules: decoder, which decodes SSTables and extracts KV pairs; merger, which performs key ordering and merging; and encoder, which re-encodes KV pairs into the SSTable structure. The architecture of the acceleration kernel is illustrated in Figure 9. Similar to existing FPGA-based compaction solutions, our design follows a dataflow-driven model that exploits the inherent streaming nature of compaction. The three modules are interconnected via FIFOs, and computation proceeds at the granularity of individual KV pairs. Data availability within the FIFOs triggers the execution of subsequent stages, enabling a deeply pipelined execution pattern.

During execution, each decoder is responsible for one input SSTable. It begins by parsing the footer and index block to locate all data blocks, which are then loaded into BRAM to reduce redundant memory accesses. Once a KV pair is decoded, its key is pushed into the FIFO connected to the merger, while the full KV pair is sent to the encoder FIFO. The merger maintains a BRAM buffer for each input SSTable, compares the buffered keys, and selects the smallest key among them. It then forwards the corresponding SSTable identifier to the encoder and fetches the next key from the associated decoder FIFO. Based on the merger's selection, the encoder retrieves the corresponding KV pair, re-encodes it into SSTable format, and updates the index and other metadata.

To address the memory constraints, we maximize BRAM utilization to support efficient data buffering and streaming. To improve DRAM bandwidth efficiency, the decoder and encoder modules utilize stream reshaping to aggregate fragmented memory requests into burst-aligned accesses matching the AXI interface width. We further enhance intra-kernel per-

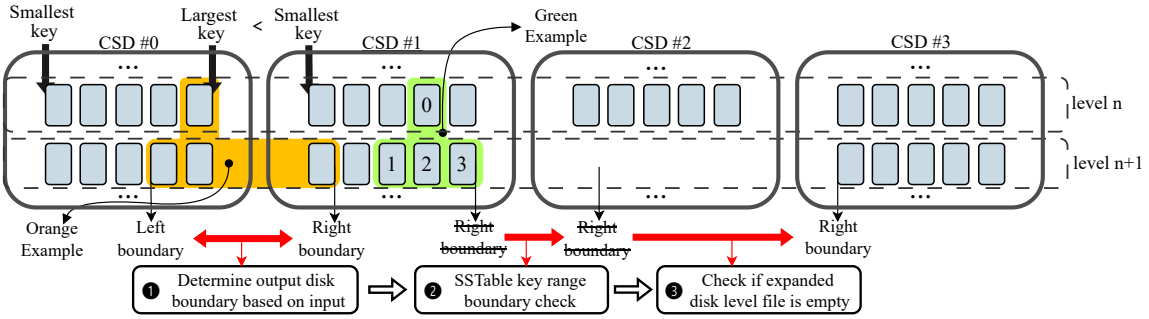


Fig. 10. CSD-Friendly Data Partitioning Scheme.

formance by widening internal data paths, deepening pipelines, and carefully overlapping computation with memory transfers. These optimizations allow a single CU to more effectively exploit the available memory bandwidth and sustain high throughput under the DRAM constraint.

When the host activates the compaction kernel, it also provides the number of KV pairs in each input SSTable. Upon completion, the kernel returns updated metadata to the host, including the number of KVs in the output, minimum and maximum key in every SSTable, etc. This enables the host side to update and maintain metadata structures accordingly.

This lightweight, pipelined kernel achieves efficient compaction execution under tight resource constraints in the CSD, enabling more relaxed constraints on the offloading conditions and contributing to higher system throughput. Furthermore, it maintains compatibility with the KV store metadata management on the host, ensuring consistency and correctness.

D. CSD-Friendly Data Partitioning Scheme

When deploying the LSM-tree based KV store on CSD arrays, we aim to achieve the following two objectives: (i) for compaction executed on the host, expose sufficient disk-level parallelism to fully utilize the aggregate I/O bandwidth of multiple CSDs; and (ii) for compaction offloaded to CSDs, minimize inter-device data transfers, as excessive cross-device traffic hampers the efficient use of each CSD's internal data transfer resources. To meet these requirements, we propose a CSD-Friendly Data Partitioning Scheme, which consists of two main components: data placement and load balancing.

Our data placement strategy partitions SSTables within each level and distributes them across multiple CSDs. The partitioning scheme employed at each level is dictated by the corresponding data layout strategy. In LSM-based key-value stores, SSTables at each level typically adhere to one of two layout strategies: Tiering or Leveling.

For levels adopting the **Tiering** strategy, we employ a round-robin placement scheme to distribute SSTables within the level. Tiering allows overlapping key ranges among SSTables in the same level, which improves write efficiency but significantly increases the cost of a single compaction. Consequently, higher levels often use Tiering layouts. However, as discussed in Section III, compactions at higher levels are not suitable for offloading to CSDs, and are scheduled to the host side. Therefore, for Tiering levels, our design objective

TABLE I
NOTATIONS USED IN THE ILP MODEL

Parameters	Description
n	Number of CSDs
m	Number of output SSTables
a_i	Number of SSTables in the output level of the i-th CSD
d_i	Number of SSTables written to the i-th CSD
l, r	Valid range of output CSDs

is to enable greater disk-level parallelism on the host side to maximize aggregate bandwidth.

Ideally, we should evenly distribute both input and output SSTables across all devices for each compaction, thereby maximizing disk-level parallelism and fully utilizing the aggregate bandwidth. Achieving this optimal distribution, however, requires workload-aware placement that considers the current KV distribution, the load status of each device, and the input set of each compaction when deciding output destinations. While theoretically appealing, this approach introduces substantial computational overhead, especially since compactions in Tiering levels often involve a large number of SSTables.

In this work, we adopt the round-robin placement scheme, which achieves near-uniform distribution with negligible overhead. Even under imbalanced workloads, the large compaction granularity and natural redistribution effect of round-robin are sufficient to maintain balanced data placement, providing a practical and efficient solution that approaches the benefits of optimal load balancing at a fraction of the complexity.

For levels adopting the **Leveling** strategy, we determine SSTable placement based on their key ranges, ensuring ordered partitions and minimizing overlaps. In these levels, the CSD collaborates with the host to execute compaction operations. The placement policy is carefully designed to minimize inter-disk data transfers when compactions are offloaded to the CSD, thereby reducing I/O overhead and improving overall efficiency. As illustrated in Figure 10, this key range-based partitioning ensures both intra-CSD and inter-CSD SSTables in each Leveling-based level maintain a globally sorted order. Under this arrangement, compaction remains the sole operation that may change the distribution of SSTables across CSDs.

To maintain key range consistency during compaction, the data placement strategy incorporates a CSD assignment mechanism that dynamically determines the output file destinations

(CSD IDs) based on the input SSTables, while preserving the global key order in corresponding level. As illustrated in Figure 10, this mechanism proceeds in three stages:

1) **Initial Output Range Estimation**: The output CSD range is estimated based on the file paths of the input SSTables at the output level. For example, in Figure 10, the victim SSTable at the output level in the orange example is stored on CSD #0 and CSD #1; therefore, the initial output CSD range is $[0, 1]$.
2) **Range Expansion**: If an input SSTable at the output level holds the smallest or largest key range among the SSTables on its level-CSD, the output range is expanded to adjacent CSDs accordingly. For instance, in Figure 10, the SSTable labeled 3 in the green example has the largest key range among those stored on CSD #1. Consequently, the right boundary of the output CSD range is extended by one, resulting in a range of $[1, 2]$.
3) **Boundary Verification**: If range expansion is triggered, the process continues until either a non-empty neighboring level is encountered or the physical disk boundary is reached. For example, in Figure 10, following the second expansion step in the green example, the corresponding level on CSD #2 is empty, prompting further extension of the right boundary to CSD #3.

To balance both data volume and workload across different CSD devices, CSD-CoKV also includes a load balancing scheme. The load balancing scheme is used to determine the number of SSTables to be written to each CSD after determining the valid range of output CSDs. The process is defined as an Integer Linear Programming (ILP) problem based on constraints and optimization objectives derived from system conditions (as shown in Table I).

ILP Constraints:

- Each valid output CSD must receive a non-negative number of files: $d_x \geq 0 \quad (x \in \{0, 1, ..., n\})$
- The total number of output files must equal the required number: $\sum_{i=l}^{r} d_i = m$
- No files are assigned outside the valid output range: $d_j = 0 \quad \text{for } j < l \text{ or } j > r$

Optimization Objective:

To ensure both data and workload are evenly distributed across the CSDs, we define the objective function based on the squared difference of SSTable counts between adjacent partitions. The optimization objective is to ensure both balanced data and workload distribution across all disks and to prevent localized imbalances or directional skews within the CSDs. For SSTable count a_i (existing) and d_i (to be written), the target is:

$$\min \sum_{i=0}^{n-1} \left(\left(\sum_{j=0}^{i-1} \frac{a_j + d_j}{i} \right) - \left(\sum_{k=i}^{n-1} \frac{a_k + d_k}{n - i} \right) \right)^2$$

By employing randomized sampling to search for near-optimal solutions, the ILP problem can be efficiently approximated with minimal computational overhead. By solving the ILP, we can determine a near-optimal output file distribution across the CSDs. Hence, our CSD-Friendly Data Partitioning Scheme minimizes inter-disk data movement during compaction and ensures balanced data volume and workload across the CSD array.

V. Evaluation

In this section, we first provide the experimental setup, and then provide the evaluation results and detailed analysis.

A. Experimental setup

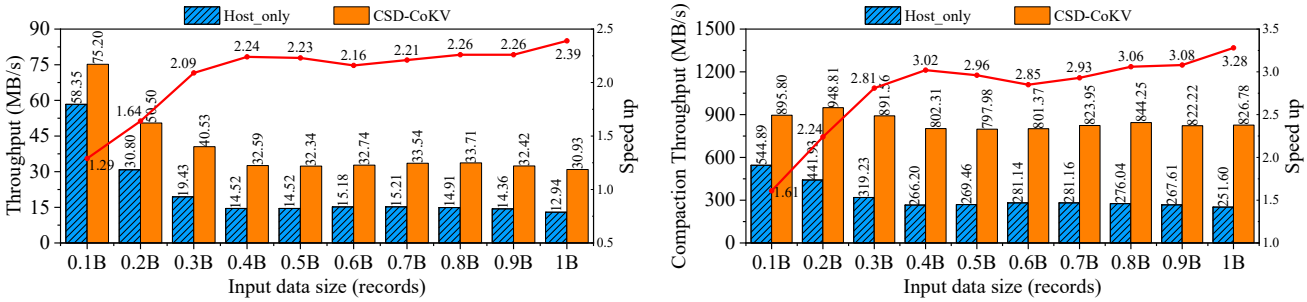
We have developed a fully functional prototype of CSD-CoKV, which is built upon RocksDB [2], a widely adopted LSM-tree key-value storage engine. For the CSD devices, we utilize commercial off-the-shelf Samsung SmartSSDs [20]. CSD-CoKV involves light changes to the existing system (only about 3700 lines of C++ code), which are mainly in the components for compaction job and column family data in RocksDB. For the CSD acceleration kernel, we developed the acceleration kernel using Xilinx Vitis HLS [30] and implemented the control of the acceleration kernel based on OpenCL. We make the prototype publicly available [31].

For performance evaluation, we use a server equipped with a 36-core Intel(R) Core(TM) i9-10980XE CPU @ 3.00GHz and 64GB of DDR4 3200 MHz memory. Four Samsung SmartSSDs are connected to the server via an NVMe expansion card [32], utilizing the PLX8747 switch chip. Each SmartSSD integrates a Xilinx Kintex UltraScale+ KU15P FPGA and is equipped with 4GB of DRAM. The SmartSSDs communicate through PCIe 3.0 $\times 4$ interfaces. The server operates on Ubuntu 20.04.6 LTS with kernel version 5.15.0-107-generic. OpenCL 2.1.0 and Xilinx Runtime (XRT) 2.14.354 are used to manage the SmartSSDs, while FPGA kernels are developed using Vitis HLS v2022.2.

All experiments are conducted based on RocksDB version 9.0.0. Since our optimizations are compatible with existing software-level optimizations, we use RocksDB which is configured with four compaction threads on the host side (named as Host-only) as our baseline. For CSD-CoKV, we enable the offloading functionality of CSD and other techniques described in this paper. CSD-CoKV is also configured with four host-side compaction threads, and additionally, one compaction thread is assigned per CSD to handle offloaded jobs. For the workload, we use RocksDB's built-in db_bench tool to issue KV requests with different distributions and sizes.

B. Overall Performance

In this section, we employ the db_bench random filling benchmark to evaluate the system overall performance. We first populate the system by inserting one billion key-value pairs with a fixed key size of 24 bytes and value size of 1024 bytes. Then we issue different volumes of data to test the system performance. For the evaluation, we primarily measured the system throughput and compaction throughput under varying amounts of write data.



(a) System Throughput. (b) Compaction throughput.

Fig. 11. Performance comparison with different data volumes.

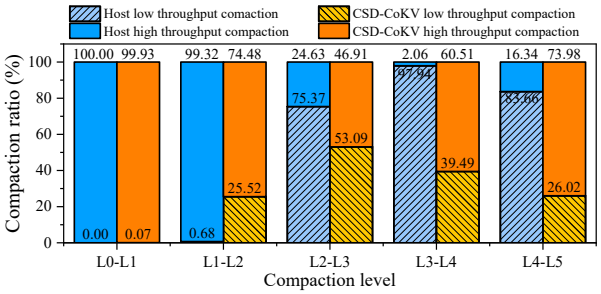


Fig. 12. Proportion of low-throughput compactions in each level comparisons, with 150 MB/s as the threshold for distinguishing low and high throughput.

As shown in Figure 11(a), the overall system write throughput is compared under different data volumes. In all cases, CSD-CoKV achieves significant performance improvements. In terms of throughput, CSD-CoKV increases the write throughput from 17.1 MB/s to 36.7 MB/s, resulting in an overall performance improvement of 2.1×. Moreover, as the amount of write data increases, the system throughput improvement provided by CSD-CoKV also grows progressively. After writing 100 million data entries, CSD-CoKV increases the write throughput from 12.94 MB/s to 30.93 MB/s, achieving the highest performance improvement of 2.39×.

We also measured the compaction throughput under different write data volumes, as shown in Figure 11(b). Similarly, CSD-CoKV consistently improves the compaction throughput across all write scenarios. The trend is analogous to that of the overall throughput: as the amount of write data increases, the performance improvement of CSD-CoKV in compaction becomes increasingly significant. For example, when the write data volume reaches 100 million, the compaction throughput of the host-only system is 251.83 MB/s, while CSD-CoKV accelerates it to 826.78 MB/s, achieving a 3.28× improvement. As the write data volume increases, the number of compactions required by the KV Store also grows. The host-only system, due to its inefficient compaction, incurs more frequent write stalls, lowering system performance. In terms of compaction throughput, CSD-CoKV improves it from 291.98 MB/s to 837.17 MB/s, resulting in a 2.86× enhancement.

Additionally, we tallied the occurrences of inefficient compactions in both systems, as mentioned in Section III and illustrated in Figure 12. In the host-only system, inefficient compactions dominate in lower-level compactions (L2-L3, L3-L4, and L4-L5). In contrast, CSD-CoKV effectively mitigates the prevalence of inefficient lower-level compactions. Therefore, CSD-CoKV can effectively address the issue of low throughput in lower-level compactions while simultaneously improving overall system throughput.

C. Sensitive Analysis

In this subsection, we provide system sensitivity analysis by varying configurations such as key sizes, value sizes, and the number of host-side compaction threads.

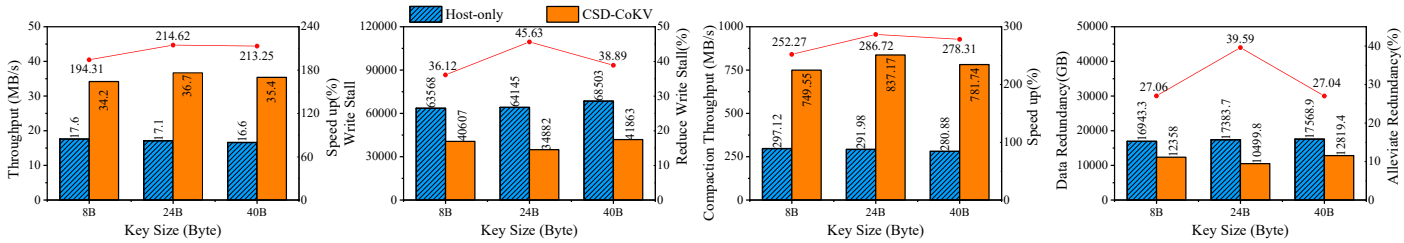
1) Key Size: We conducted experiments using *db_bench* with three key sizes, 8 bytes, 24 bytes, and 40 bytes, while fixing the value size at 1024 bytes. Each experiment involved inserting one billion uniformly distributed KV pairs. The results are shown in Figure 13.

Figure 13 (a) reports the system throughput comparison across different key lengths. CSD-CoKV consistently improves system throughput, achieving gains of 194.31%, 214.62%, and 213.25% for the respective key sizes. To provide deeper insights, Figure 13 (b) presents the number of write stalls. Compared to the host-only baseline, CSD-CoKV reduces stalls by 36.12%, 45.63%, and 38.89% for 8-byte, 24-byte, and 40-byte keys, respectively. The reduction in write stalls directly contributes to the overall throughput improvements, with the maximum benefit again observed at 24-byte keys (45.63% fewer stalls).

Figure 13 (c) provides the compaction throughput comparison results. CSD-CoKV achieves improvements of 252.27%, 286.72%, and 278.31% for 8-byte, 24-byte, and 40-byte keys, respectively. For the host-only system, compaction throughput slightly decreases as key size grows due to increased sorting and merging overhead. In contrast, CSD-CoKV achieves the highest compaction throughput gain with 24-byte keys. When keys are shorter (8 bytes), the compaction speedup is relatively limited, as shorter keys would reduce parallelism.

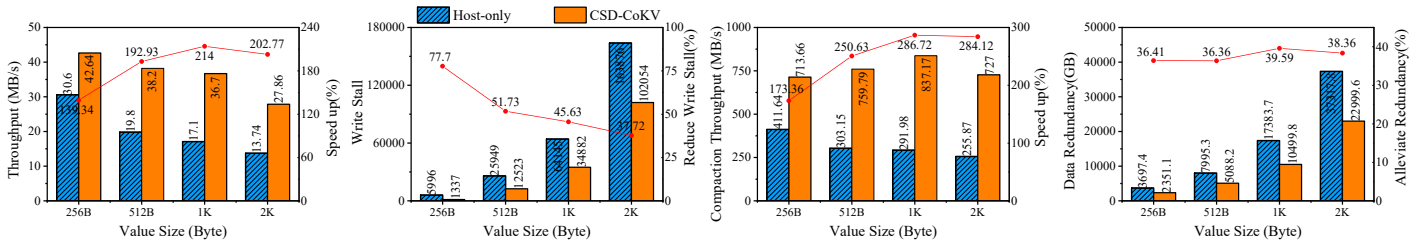
Figure 13 (d) shows the data transfers between the host and storage. By offloading compaction to storage device, CSD-CoKV reduces host–device data movement by 27.06%, 39.59%, and 27.04% for the three key sizes, with the largest reduction reaching 6883.9 GB.

2) Value size: We further employ *db_bench* random filling benchmark to issue one billion KV pairs for the performance evaluation with different value sizes (256B, 512B, 1KB, and 2KB). The results are shown in Figure 14.



(a) System Throughput. (b) Number of write stalls. (c) Compaction throughput. (d) Data redundancy.

Fig. 13. The performance under different key sizes (8B, 24B, and 40B).



(a) System Throughput. (b) Number of write stalls. (c) Compaction throughput. (d) Data redundancy.

Fig. 14. The performance under different value sizes (256B, 512B, 1KB, and 2KB).

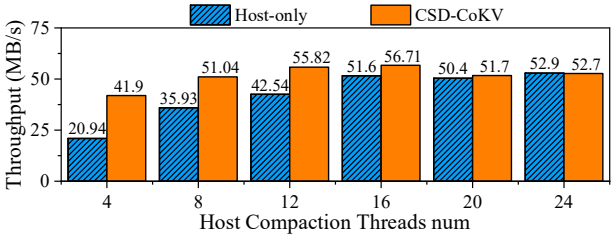


Fig. 15. Throughput comparison under different host threads.

Figure 14 (a) shows that compared with the host-only system, CSD-CoKV improves system overall throughput by 1.39×, 1.92×, 2.14×, and 2.02× for 256B, 512B, 1KB, and 2KB values, respectively. The performance gain increases as value size grows from 256B to 1KB, reaching its peak at 1KB (from 17.1 MB/s to 36.7 MB/s), but slightly declines for 2KB values due to larger compaction files, which limit offloading efficiency. Figure 14 (b) further shows that CSD-CoKV significantly reduces stall occurrences, e.g. by 77.7% (from 5996 to 1337) for 256B values. The reduction ratio decreases as value size grows, since larger values increase total data volume and I/O pressure.

Figure 14 (c) shows the compaction throughput comparison. For 1KB values, CSD-CoKV increases the compaction throughput from 291.98 MB/s to 837.17 MB/s, achieving a 2.86× speedup. This trend aligns with overall throughput improvements, indicating that in-storage compaction helps maintain the LSM-tree structure more efficiently and reduces write stalls. Moreover, as shown in Figure 14 (d), performing compaction inside the CSD also reduces host–device data movement by about 37% across all value sizes, with the largest reduction of 14,080.2 GB at 2KB values. This reduction alleviates interconnect contention and further contributes to overall performance gains.

3) Host threads: We further vary the number of host threads (4, 8, 12, 16, 20 and 24) to test how the system performance changes of both the host-only system and the CSD-CoKV design. They are configured with the same number of host threads. We use *db_bench* to randomly write 500 million KV pairs with key size of 24 bytes and value size of 1KB. The system throughput comparison is shown in Figure 15.

Overall, as the number of host threads increases, the performance improvement of CSD-CoKV on the system gradually diminishes. For the host-only system, throughput increases steadily up to 16 threads (reaching 51.6 MB/s), but then shows only marginal gains (50.4 MB/s at 20 threads, and 52.9 MB/s at 24 threads), indicating a clear saturation of performance scalability. In contrast, CSD-CoKV achieves its peak throughput at 16 threads (56.71 MB/s), but its performance decreases when more host threads are added (dropping to 51.7 MB/s at 20 threads and 52.7 MB/s at 24 threads). This degradation suggests that excessive host threading starts to compete with the CSD for I/O resources, and the increased CPU-side contention leads to lower per-thread efficiency, ultimately harming overall throughput.

Notably, when CSD-CoKV uses only 4 host threads, the write throughput is comparable to that of the host-only system with 12 threads. With this configuration, CSD-CoKV successfully reduces the host CPU utilization from 483.82% to 293.97%. Despite the observed performance decline under high thread counts, CSD-CoKV still provides the best throughput across all tested configurations at 16 threads, demonstrating its effectiveness in improving system performance.

D. Scalability

To assess the performance scalability of the CSD-Friendly Data Partitioning Scheme under a CSD array, we evaluated CSD-CoKV across configurations ranging from one to four CSDs using *db_bench*. In the single-CSD configuration, we allocated one host thread. To ensure linear scalability with respect to computational resources, we added one host thread for each additional CSD. We inserted 500 million key-value pairs into the database, with each key measuring 24 bytes and

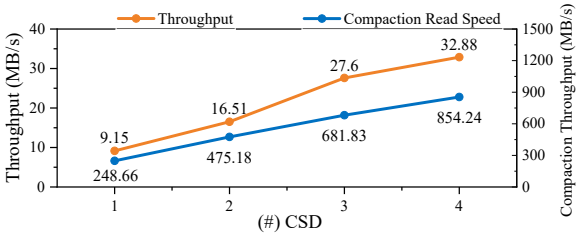


Fig. 16. Scalability with number of CSDs.

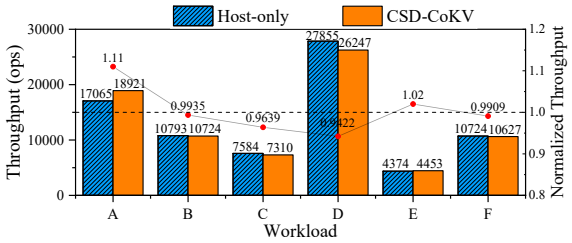


Fig. 17. Throughput of CSD-CoKV and host-only under YCSB workloads.

each value 1 KB in size. Once performance had stabilized, we measured both the write throughput and the compaction read speed for all four configurations, as presented in Figure 16.

We observed that as the number of CSDs increased, the overall compaction throughput of the system scaled nearly linearly. Similarly, the overall system throughput also improved with the addition of more CSDs. These results demonstrate that CSD-CoKV exhibits strong scalability, with compaction performance increasing linearly as additional CSDs are incorporated. Consequently, integrating more CSDs can significantly enhance the system's throughput capacity, presenting a promising solution for high-performance key-value stores.

E. Macro benchmark with YCSB

To evaluate system performance in realistic scenarios, we conduct macro-benchmark experiments using YCSB. The database is preloaded with 200 million KV pairs to emulate large-scale deployments, with 24-byte keys and 1024-byte values. YCSB defines six representative workloads with different operation mixes and key distributions: Workload A is balanced (50% reads, 50% updates), Workload B is read-heavy (95% reads, 5% updates), and Workload C is read-only. Workload D consists mostly of reads with occasional inserts (95/5) under a "latest" key distribution. Workload E is dominated by scans (95% scans, 5% inserts), while Workload F mixes reads and read-modify-write operations (50/50). All workloads except D follow a Zipfian key access pattern. We compare the baseline host-only execution with CSD-CoKV, and the normalized throughput results are shown in Figure 17.

CSD-CoKV is primarily optimized for write-intensive workloads by reducing write amplification through more efficient compaction. This optimization has a notable impact in Workload A (balanced), where CSD-CoKV achieves an 11% throughput improvement. The gain is driven by faster compaction and reduced I/O overhead, although the overall improvement is partially limited by the relatively constrained read throughput in such large-scale settings.

For read-intensive workloads, the impact of CSD-CoKV is small. In Workloads B, C, and F, throughput changes

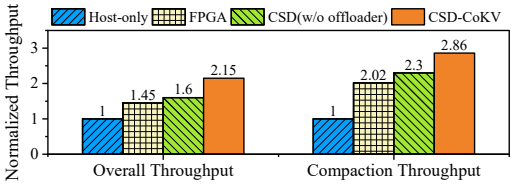


Fig. 18. Normalized throughput comparison with SOTA.

are minimal, with reductions of 0.75%, 3.61%, and 0.91%, respectively, while in Workload E (scan-heavy), throughput improves by about 2%. All these variations are within 5%, indicating that CSD-CoKV introduces only minor impact on read performance. For workload D, where the access pattern follows a Latest distribution, host memory can buffer the SSTables required by transactions more effectively, thereby posing challenges to data locality in CSD-offloaded computation. Even in this case, the performance degradation of CSD-CoKV is limited to 5.75%, which remains acceptable.

F. Compared with SOTA

To further validate the effectiveness of our design, we compare it with representative accelerator-based configurations. Since our design is orthogonal to existing software-level optimizations, we only add two configurations: (i) FPGA, which uses the same model and number of external FPGA devices to implement existing FPGA-based solutions, and (ii) CSD (w/o offloader), which disables the semantic-aware offloader in CSD-CoKV while keeping all other settings unchanged. For consistency, all configurations are evaluated with four host threads. The normalized overall and compaction throughput are reported in Figure 18.

Compared with the host-only baseline, the FPGA configuration achieves about 45% improvement in overall throughput and 102% improvement in compaction throughput, benefiting from additional compute resources. However, the performance gain is constrained by repeated host-device data transfers and system interconnect contention. The CSD (w/o offloader) configuration yields a more substantial benefit, delivering approximately 60% higher overall throughput and 130% higher compaction throughput than host-only, indicating the potential of in-storage acceleration. Finally, CSD-CoKV further improves performance to 115% higher overall throughput and 186% higher compaction throughput. With a 34% overall and 24% compaction gain over CSD (w/o offloader), the effectiveness of our semantic-aware offloader in selecting the right compactions for in-storage execution is clearly demonstrated. Compared with the FPGA configuration, CSD-CoKV leverages independent internal data paths and semantic-aware compaction selection, achieving 48% improvement in overall throughput and 42% improvement in compaction throughput.

G. Overhead Analysis

We first analyze the energy and monetary overhead of incorporating CSDs in our design. Based on the experiments in Section V-B, we measured the power consumption of both the Host-only and CSD-CoKV configurations using PowerTop [33] and XRT [34]. We then calculated the average energy

cost per KV operation by considering the dynamic power of the CPU and CSD. In the Host-only configuration, the average cost is 7.617 mJ/operation, while in CSD-CoKV it is reduced to 6.81 mJ/operation, representing a 10.6% energy reduction. Although using CSDs introduces extra overhead from the accelerator unit and host-side management, CSD-CoKV effectively offsets this cost by offloading compaction to the storage device, thereby reducing the high host–disk data transfer overhead and leveraging efficient accelerator kernels to achieve high throughput under lower energy consumption.

However, current commercial CSDs are still in an early stage of academic and prototype development, and their costs have not yet benefited from large-scale production. In our system, each CSD costs approximately $2500, compared to only $400 for a commodity SSD of the same capacity. By contrast, the FPGA accelerator embedded in a CSD is equivalent to a standalone FPGA chip costing about $100, suggesting that future CSD deployments have significant potential for reduced cost once mass production becomes feasible.

We then analyze the computational overhead of ILP-based load balancing (Section IV-D). Due to the key-range partitioning constraints of leveling, effective CSD paths typically involve only two devices, allowing ILP to be solved in less than 1 ms via exhaustive search in most cases. To handle extreme cases, CSD-CoKV employs randomized sampling. In such scenarios, when using 8 CSD devices and 7 input files with a traversal step of 2, the ILP solution completed in 19 ms; with 12 CSD devices, the solution still completed within 54 ms. Even under high compaction throughput, the impact of ILP solving accounts for less than 1% of a single compaction's latency. Therefore, the ILP-based scheduling in CSD-CoKV introduces negligible computational overhead.

VI. Related work

Prior work has explored various approaches to optimizing compaction in LSM-tree–based KV stores, spanning both software and hardware.

Software-Level Optimizations. At the software level, several techniques mitigate compaction overhead via adaptive scheduling and data structure improvements. ADOC [6] dynamically adjusts the number of compaction threads and flush batch size. SILK [25] and SILK+ [35] reduce latency spikes through latency-aware scheduling, with SILK+ further enhancing concurrency and robustness via multiple compaction tasks and dynamic I/O allocation. LSM-Bash [36] reorganizes the LSM-tree into a bush-like structure, lowering compaction interference and improving throughput.

Key-value separation is another effective approach for reducing write amplification, especially for workloads with large values. Systems such as DiffKV [37], SpacKV [38], and SineKV [39] store large values separately from the LSM-tree, keeping only keys or metadata within the tree. This reduces the volume of data involved in compactions, minimizing the frequency and cost of rewriting large values, thereby improving write throughput and reducing write amplification.

Hardware-Level Optimizations. Recent efforts have explored offloading compaction to hardware accelerators, including FPGAs [13], [14], [40], GPUs [15], [16], [41], [42], and CSDs [18], [21], [23], [43].

FPGA-based solutions attach FPGAs to the host via PCIe and leverage their reconfigurable logic and parallelism. [13] and [14] fully utilize FPGA resources for compaction, while [40] employs multi-unit pipelines with dynamic scheduling for large-scale compactions. However, I/O transfers over the shared system interconnect remain a bottleneck.

GPU-based approaches exploit the massive parallelism of GPGPUs. LUDA [41] and gLSM [15] offload all compaction computations to GPUs, improving throughput. GPUKV [42] integrates GPU computation with KVSSD using PCIe P2P communication, reducing CPU involvement and data movement latency. RGKV [16] further optimizes GPU memory layout, parallel merge-sort, and GPUDirect transfers to enhance LSM-tree compaction efficiency.

CSD-based systems offload compaction directly to storage devices. NKV [21] utilizes native CSD to process compaction tasks close to data, reducing host-side I/O and compute overhead. Similarly, [43] and [23] explore the coordination of compaction tasks across multiple storage-side processing units, but these approaches are typically evaluated using emulation platforms. Moreover, their offloading approaches are straightforward, merely offloading all compactions to CSDs for computation, which may even decrease the system performance. AegonKV [18] employs SmartSSD to offload garbage collection under KV separation, thereby achieving a balanced trade-off among throughput, tail latency, and disk utilization.

VII. Conclusion

In this paper, we present CSD-CoKV, a novel framework that harmonizes CSDs with the host to tackle the performance challenges of LSM-tree-based key-value stores. By carefully analyzing the inefficiencies in naive in-storage compaction offloading, we design a semantic-aware offloader that intelligently selects compaction jobs suitable for CSD execution, a lightweight high-speed compaction kernel for in-storage computation, and a CSD-friendly data partitioning scheme that optimizes data placement and workload balance across CSD arrays. Experimental results demonstrate that CSD-CoKV effectively reduces unnecessary data movement over the system interconnect, alleviates host-side resource contention, and significantly improves compaction efficiency. Compared to a traditional host-only configuration, CSD-CoKV achieves up to 2.39× higher write throughput and up to 3.28× faster compaction speed while maintaining strong scalability.

VIII. Acknowledgment

We thank the reviewers for their constructive suggestions that improved this paper. The work described in this paper is supported by the Key R&D Program of Shandong Province under Grant 2023CXPT002, the National Natural Science Foundation of China under Grant 624B2091, 62272271 and the Taishan Scholars Program under Grant tsqn202408009.

IX. AI-Generated Content Acknowledgment

This paper employed generative artificial intelligence tools, including ChatGPT [44] and Kimi [45], to assist in grammar checking, typo correction, and language expression optimization throughout the entire manuscript. These tools were used only to improve the clarity and readability of the writing.

No AI system contributed to the creation of technical content, ideas, algorithms, analyses, experiments, or figures. All scientific contributions and substantive writing were solely developed by the authors. All AI-assisted edits were reviewed and verified by the authors, who take full responsibility for the final content of this paper.

References

[1] P. O'Neil, E. Cheng, D. Gawlick, and E. O'Neil, "The log-structured merge-tree," *Acta Informatica*, vol. 33, no. 4, pp. 351–385, 1996.

[2] Meta, "Rocksdb," https://rocksdb.org.

[3] J. Dean and S. Ghemawat, "Leveldb: A fast persistent key-value store," https://github.com/google/leveldb.

[4] A. S. Foundation, "Apache cassandra," https://cassandra.apache.org/.

[5] ——, "Apache hbase," https://hbase.apache.org/.

[6] J. Yu, S. H. Noh, Y.-r. Choi, and C. J. Xue, "Adoc: Automatically harmonizing dataflow between components in log-structured key-value stores for improved performance," in *21st USENIX Conference on File and Storage Technologies (FAST)*, 2023, pp. 65–80.

[7] L. Lu, T. S. Pillai, H. Gopalakrishnan, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "Wisckey: Separating keys from values in ssd-conscious storage," *ACM Transactions On Storage*, vol. 13, no. 1, pp. 1–28, 2017.

[8] S. Wu, Z. Wang, S. Wang, J. Chen, C. Du, J. Zhang, K. Zhou, and B. Mao, "Bl-tree: The best of both worlds by combining b+-tree on top and lsm-tree on bottom," in *IEEE 41st International Conference on Data Engineering (ICDE)*, 2025, pp. 1910–1923.

[9] Z. Lu, Q. Cao, H. Jiang, Y. Chen, J. Yao, and A. Pan, "Fluidkv: Seamlessly bridging the gap between indexing performance and memory-footprint on ultra-fast storage," *Proceedings of the VLDB Endowment*, vol. 17, no. 6, pp. 1377–1390, 2024.

[10] M. Cai, J. Shen, Y. Yuan, Z. Qu, and B. Ye, "Bonsaikv: Towards fast, scalable, and persistent key-value stores with tiered, heterogeneous memory system," *Proceedings of the VLDB Endowment*, vol. 17, no. 4, pp. 726–739, 2023.

[11] Q. Wei, Z. Shen, Y. Tong, Z. Jia, L. Ju, J. Chen, and B. Li, "Re-lsm: A reram-based processing-in-memory framework for lsm-based key-value store," in *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2022, pp. 1–9.

[12] Z. Chen, K. Zhang, Q. Wei, N. Su, Y. Zhang, Z. Shen, D. Yu, and L. Ju, "A reram-based processing-in-memory framework for lsm-based key-value store," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2025.

[13] T. Zhang, J. Wang, X. Cheng, H. Xu, N. Yu, G. Huang, T. Zhang, D. He, F. Li, W. Cao *et al.*, "Fpga-accelerated compactions for lsm-based key-value store," in *18th USENIX Conference on File and Storage Technologies (FAST)*, 2020, pp. 225–237.

[14] X. Sun, J. Yu, Z. Zhou, and C. J. Xue, "Fpga-based compaction engine for accelerating lsm-tree key-value stores," in *IEEE 36th International Conference on Data Engineering (ICDE)*, 2020, pp. 1261–1272.

[15] H. Sun, J. Xu, X. Jiang, G. Chen, Y. Yue, and X. Qin, "glsm: Using gpgpu to accelerate compactions in lsm-tree-based key-value stores," *ACM Transactions on Storage*, vol. 20, no. 1, pp. 1–41, 2024.

[16] H. Sun, X. Jiang, Y. Yue, and X. Qin, "Rgkv: A gpgpu-empowered compaction framework for lsm-tree-based kv stores with optimized data transfer and parallel processing," *IEEE Transactions on Computers*, 2025.

[17] Z. Yang, Y. Lu, X. Liao, Y. Chen, J. Li, S. He, and J. Shu, "λ-io: A unified io stack for computational storage," in *21st USENIX Conference on File and Storage Technologies (FAST)*, 2023, pp. 347–362.

[18] Z. Duan, H. Feng, H. Liu, X. Liao, H. Jin, and B. Li, "Aegonkv: A high bandwidth, low tail latency, and low storage cost kv-separated lsm store with smartssd-based gc offloading," in *23rd USENIX Conference on File and Storage Technologies (FAST)*, 2025, pp. 321–335.

[19] H. Jang, J. Song, J. Jung, J. Park, Y. Kim, and J. Lee, "Smart-infinity: Fast large language model training using near-storage processing on a real system," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 345–360.

[20] Samsung, "Smart ssd — ssd card — samsung semiconductor global," https://semiconductor.samsung.com/ssd/smart-ssd/.

[21] T. Vincon, A. Bernhardt, I. Petrov, L. Weber, and A. Koch, "nkv: near-data processing with kv-stores on native computational storage," in *Proceedings of the 16th International Workshop on Data Management on New Hardware (DaMoN)*, 2020, pp. 1–11.

[22] H. Sun, C. Zhao, Y. Yue, and X. Qin, "Prockstore: An ndp-empowered key-value store with asynchronous and multi-threaded compaction scheme for optimized performance," *Journal of Systems Architecture*, p. 103342, 2025.

[23] H. Sun, Q. Wang, Y. L. Yue, Y. Zhao, and S. Fu, "A storage computing architecture with multiple ndp devices for accelerating compaction performance in lsm-tree based kv stores," *Journal of Systems Architecture*, vol. 130, p. 102681, 2022.

[24] Z. Cao, S. Dong, S. Vemuri, and D. H. Du, "Characterizing, modeling, and benchmarking rocksdb key-value workloads at facebook," in *18th USENIX Conference on File and Storage Technologies (FAST)*, 2020, pp. 209–223.

[25] O. Balmau, F. Dinu, W. Zwaenepoel, K. Gupta, R. Chandhiramoorthi, and D. Didona, "Silk: Preventing latency spikes in log-structured merge key-value stores," in *2019 USENIX Annual Technical Conference (USENIX ATC)*, 2019, pp. 753–766.

[26] Q. Wei, Z. Chen, X. Chen, Y. Zhang, X. Cai, Z. Jia, Z. Shen, Y. Wang, Z. Shao, and B. Li, "A semantic-integrated lsm-tree-based key–value storage engine for blockchain systems," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 6, pp. 1794–1807, 2023.

[27] Z. Chen, B. Li, X. Cai, Z. Jia, L. Ju, Z. Shao, and Z. Shen, "Chainkv: A semantics-aware key-value store for ethereum system," *Proceedings of the ACM on Management of Data*, vol. 1, no. 4, pp. 1–23, 2023.

[28] Z. Zhao, Z. Shen, and X. Cai, "Isvda: An in storage processing accelerator for visual data analysis," in *2024 IEEE 42nd International Conference on Computer Design (ICCD)*, 2024, pp. 661–664.

[29] AMD, "SmartSSD® Computational Storage Drive," https://www.xilinx.com/publications/product-briefs/xilinx-smartssd-computational-storage-drive-product-brief.pdf, 2025.

[30] AMD, "Amd vitis hls," https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vitis/vitis-hls.html.

[31] Z. Cao, J. Yang, and S. Ma, "CSD-CoKV: Host-CSD Collaborative Offloading for High-Performance LSM-tree based KV Stores," https://github.com/caozhining/CSD-RocksDB, 2025.

[32] LinkReal, "LRNV9347L-4I PCIe 3.0 x8 to 4 Port SFF-8643 NVMe Expansion Card," http://www.linkreal.com.cn/products/LRNV9347L4I.html.

[33] fenrus75, "PowerTop," https://github.com/fenrus75/powertop.

[34] Xilinx, "XRT," https://github.com/Xilinx/XRT.

[35] O. Balmau, F. Dinu, W. Zwaenepoel, K. Gupta, R. Chandhiramoorthi, and D. Didona, "Silk+ preventing latency spikes in log-structured merge key-value stores running heterogeneous workloads," *ACM Transactions on Computer Systems*, vol. 36, no. 4, pp. 1–27, 2020.

[36] N. Dayan and S. Idreos, "The log-structured merge-bush & the wacky continuum," in *Proceedings of the 2019 International Conference on Management of Data*, 2019, pp. 449–466.

[37] Y. Li, Z. Liu, P. P. C. Lee, J. Wu, Y. Xu, Y. Wu, L. Tang, Q. Liu, and Q. Cui, "Differentiated key-value storage management for balanced i/o performance," in *2021 USENIX Annual Technical Conference (USENIX ATC)*, 2021, pp. 673–687.

[38] X. Ge, M. Lai, Y. Liu, L. Wu, Z. Zhuang, Y. Ou, Z. Chen, and N. Xiao, "Spackv: A pmem-aware key-value separation store based on lsm-tree," in *Network and Parallel Computing: 19th IFIP WG 10.3 International Conference, NPC 2022, Jinan, China, September 24-25, 2022, Proceedings*, 2022, pp. 327–339.

[39] F. Li, Y. Lu, Z. Yang, and J. Shu, "Sinekv: Decoupled secondary indexing for lsm-based key-value stores," in *IEEE 40th International Conference on Distributed Computing Systems (ICDCS)*, 2020.

[40] D. Tang, W. Wang, Y. Mao, J. Yu, T.-W. Kuo, and C. J. Xue, "Stem: Streaming-based fpga acceleration for large-scale compactions in lsm kv," in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, 2024, pp. 3893–3905.

[41] P. Xu, J. Wan, P. Huang, X. Yang, C. Tang, F. Wu, and C. Xie, "Luda: Boost lsm key value store compactions with gpus," *arXiv preprint arXiv:2004.03054*, 2020.
[42] M.-G. Jung, C.-G. Lee, D. Park, S. Park, J. Noh, W. Chung, K. Park, and Y. Kim, "Gpukv: an integrated framework with kvssd and gpu through p2p communication support," in *Proceedings of the 36th Annual ACM Symposium on Applied Computing*, 2021, pp. 1156–1164.
[43] H. Sun, B. Lou, C. Zhao, D. Kong, C. Zhang, J. Huang, Y. Yue, and X. Qin, "Asynchronous compaction acceleration scheme for near-data processing-enabled lsm-tree-based kv stores," *ACM Transactions on Embedded Computing Systems*, vol. 23, no. 6, pp. 1–33, 2024.
[44] OpenAI, "ChatGPT," https://chat.openai.com/.
[45] Moonshot AI, "Kimi Chat," https://kimi.moonshot.cn/.